

TypeScript - Principiante

Tema 06: Union types y literal types

20 ejercicios progresivos, sin soluciones, para practicar union types, literal types, restricciones de valores y narrowing.

Ejercicio 1 - Identificador flexible

Crea una variable id que pueda contener un numero o un string.

Requisitos

- Asigna primero un number.
- Despues asigna un string.
- No uses any.

Ejemplo de ejecucion

100
USR-100

Objetivo

Practicar union types basicos.

Pista

Usa | para combinar tipos.

Ejercicio 2 - Entrada de usuario

Crea una funcion que reciba un valor string o number y lo muestre.

Requisitos

- Tipa el parametro con una union.
- Prueba ambos tipos.

Ejemplo de ejecucion

Valor: 25
Valor: TypeScript

Objetivo

Usar unions en parametros.

Pista

Un parametro puede aceptar mas de un tipo.

Ejercicio 3 - Estado restringido

Crea un tipo Status que solo acepte pending, approved o rejected.

Requisitos

- Crea varias variables validas.
- Incluye como comentario un valor invalido.

Ejemplo de ejecucion

Estado: approved

Objetivo

Practicar literal types.

Pista

No uses string general.

Ejercicio 4 - Tema visual

Crea una variable theme que solo permita light o dark.

Requisitos

- Cambia el valor durante la ejecucion.
- Muestra un mensaje segun el tema.

Ejemplo de ejecucion

Tema activo: dark

Objetivo

Practicar literales de string.

Pista

Los valores permitidos forman parte del tipo.

Ejercicio 5 - Prioridad

Define una prioridad que solo acepte low, medium o high y crea una funcion que la reciba.

Requisitos

- Maneja las tres posibilidades.
- Devuelve un mensaje.

Ejemplo de ejecucion

high -> Atender inmediatamente

Objetivo

Combinar literales con funciones.

Pista

Un switch puede ayudarte.

Ejercicio 6 - Resultado numerico o mensaje

Crea una funcion que devuelva un number cuando una operacion sea valida o un string cuando exista un error.

Requisitos

- Declara el union type de retorno.
- Prueba ambos resultados.

Ejemplo de ejecucion

Resultado: 20

Error: division entre cero

Objetivo

Practicar unions en retornos.

Pista

El retorno puede ser number | string.

Ejercicio 7 - Narrowing con typeof

Crea una funcion que reciba string | number. Si recibe string, devuelve su longitud; si recibe number, devuelve su doble.

Requisitos

- Usa typeof.
- No hagas conversiones innecesarias.

Ejemplo de ejecucion

Hola -> 4

10 -> 20

Objetivo

Practicar type narrowing.

Pista

Despues de typeof TypeScript conoce el tipo concreto.

Ejercicio 8 - Boolean o texto

Crea una funcion que reciba boolean | string y genere una descripcion apropiada.

Requisitos

- Distingue ambos tipos.
- Tipa el retorno como string.

Ejemplo de ejecucion

true -> Activado

manual -> manual

Objetivo

Reforzar narrowing.

Pista

Comprueba el tipo antes de usar operaciones especificas.

Ejercicio 9 - Metodo de pago

Define PaymentMethod con cash, card y transfer.

Requisitos

- Crea una funcion para procesar cada metodo.
- No permitas otros valores.

Ejemplo de ejecucion

Pago mediante card

Objetivo

Modelar opciones cerradas.

Pista

Los literal types son utiles para estados y opciones.

Ejercicio 10 - Tamano de componente

Define un tipo Size con small, medium y large. Crea una funcion que devuelva un numero segun el tamano.

Requisitos

- Cada literal debe producir un valor diferente.
- Maneja todos los casos.

Ejemplo de ejecucion

medium -> 32

Objetivo

Relacionar literales con comportamiento.

Pista

Puedes usar switch.

Ejercicio 11 - Id de usuario

Define un type UserId que acepte string | number y usalo en un objeto User.

Requisitos

- Crea usuarios con ambos tipos de id.
- No dupliques la union en cada lugar.

Ejemplo de ejecucion

1 - Ana

USR-2 - Carlos

Objetivo

Reutilizar union types mediante alias.

Pista

Un type alias evita repetir tipos complejos.

Ejercicio 12 - Estado de pedido

Crea un objeto Order cuyo status solo permita created, paid, shipped o cancelled.

Requisitos

- Crea al menos cuatro pedidos.
- Filtra pedidos por un status recibido como parametro.

Ejemplo de ejecucion

Pedidos shipped: 2

Objetivo

Usar literales dentro de objetos y colecciones.

Pista

Tipa tambien el parametro del filtro.

Ejercicio 13 - Nivel de acceso

Define roles admin, editor y viewer. Crea una funcion que devuelva las acciones permitidas para cada rol.

Requisitos

- El rol debe estar restringido.
- Devuelve string[].

Ejemplo de ejecucion

editor -> edit, view

Objetivo

Practicar literales como reglas de dominio.

Pista

Cada rol puede mapearse a diferentes acciones.

Ejercicio 14 - Direccion de ordenamiento

Define SortDirection como asc | desc y crea una funcion que ordene numeros segun esa direccion.

Requisitos

- La direccion debe tener un valor por defecto.
- Devuelve un nuevo array.

Ejemplo de ejecucion

asc: 1, 4, 8, 10

Objetivo

Combinar literal types con valores por defecto.

Pista

El comparador cambia segun la direccion.

Ejercicio 15 - Resultado discriminado

Modela un resultado que pueda ser exitoso con data o fallido con error.

Requisitos

- Ambas variantes deben tener una propiedad comun que permita distinguirlas.
- No uses any.
- Crea una funcion que procese ambas.

Ejemplo de ejecucion

Success: usuario encontrado

Error: usuario no encontrado

Objetivo

Introducir discriminated unions.

Pista

Una propiedad literal como success o type puede discriminar variantes.

Ejercicio 16 - Notificacion

Modela notificaciones email, sms y push, donde cada tipo necesite datos diferentes.

Requisitos

- Usa una union de objetos.
- Cada variante debe tener un literal que la identifique.
- Procesalas con narrowing.

Ejemplo de ejecucion

Email enviado a user@test.com

Objetivo

Practicar unions de objetos.

Pista

El discriminante permite acceder con seguridad a propiedades especificas.

Ejercicio 17 - Respuesta de API

Modela una respuesta loading, success o error.

Requisitos

- loading no necesita datos.
- success contiene data.
- error contiene message.
- Crea una funcion que muestre el estado.

Ejemplo de ejecucion

Cargando...

Datos recibidos

Error: servidor no disponible

Objetivo

Modelar estados comunes de aplicaciones frontend.

Pista

Cada estado puede tener una estructura distinta.

Ejercicio 18 - Filtro configurable

Crea una funcion que filtre productos por estado active | inactive | all.

Requisitos

- all devuelve todos.
- Los otros valores filtran.
- El parametro solo acepta esos tres literales.

Ejemplo de ejecucion

Filtro active: 5 productos

Objetivo

Aplicar literales a configuracion de funciones.

Pista

Evita usar boolean cuando existen mas de dos estados utiles.

Ejercicio 19 - Evento de aplicacion

Modela eventos login, logout y purchase. Cada evento debe contener datos apropiados para su tipo.

Requisitos

- Usa discriminated unions.
- Crea una funcion que procese todos los eventos.
- Maneja cada variante de forma segura.

Ejemplo de ejecucion

Login: usuario 10

Purchase: \$850

Objetivo

Practicar modelado de eventos tipados.

Pista

El tipo del evento puede actuar como discriminante.

Ejercicio 20 - Sistema de operaciones

Crea una coleccion de operaciones que puedan ser deposit, withdrawal o transfer. Cada tipo debe contener solamente los datos que necesita.

Requisitos

- Usa una union discriminada.
- Deposito requiere amount.
- Retiro requiere amount.
- Transferencia requiere amount y destinationAccount.
- Crea una funcion para procesar cada operacion.
- Rechaza cantidades invalidas.
- No uses any.
- Maneja todos los tipos de operacion.

Ejemplo de ejecucion

Deposit: \$1000

Transfer: \$500 -> ACC-20

Objetivo

Integrar union types, literal types, objetos y narrowing.

Pista

Modela primero cada variante y luego crea la union.